DIGITAL EGYPT PIONEERS INITIATIVE (DEPI)

MINISTRY OF COMMUNICATIONS AND INFORMATION TECHNOLOGY

DEVOPS TRACK

FINAL PROJECT DOCUMENTATION

INTELLIGENT TRAFFIC MANAGEMENT DASHBOARD

A DevOps-Based Solution for Real-Time Traffic Telemetry Ingestion, Monitoring, and Automated Deployment

Tools: Git | Jenkins | Docker | Kubernetes | Terraform | Ansible | Nginx | Prometheus | Grafana | AWS EC2/S3

SUBMITTED TO	PREPARED BY
Eng. Mena Magdy	1. Eng. Marwan Elmadawi 2. Eng. Mohamed Mekhemar 3. Eng. Nagat Ali 4. Eng. Asmaa Hany 5. Eng. Mohamed Abdelhamed
SUBMISSION DATE	**ACADEMIC YEAR**
16 October 2026	2026 – 2027

COLLECT • DEPLOY • SCALE • MONITOR

TABLE OF CONTENTS

Project Documentation

1. Project Planning & Management	**3**
1.1 Project Proposal	3
1.1.1 Project Overview	3
1.1.2 Problem Statement	3
1.1.3 Proposed Solution	3
1.1.4 Expected Outcome	3
1.2 Project Objectives	3
1.3 Project Scope	4
1.3.1 In-Scope Activities	4
1.3.2 Out-of-Scope Activities	4
1.3.3 Assumptions and Constraints	4
1.3.4 Main Scope Deliverables	4
1.4 Project Plan and Timeline	4
1.4.1 Project Plan Overview	4
1.4.2 Project Timeline	5
1.4.3 Gantt Chart	5
1.4.4 Progress Monitoring	5
1.5 Milestones and Deliverables	5
1.5.1 Project Milestones	5
1.5.2 Deliverables and Acceptance Criteria	6
1.6 Task Assignment and Roles	6
1.6.1 Team Structure	6
1.6.2 Task Allocation	7
1.6.3 Collaboration	7
1.7 Resource Allocation	7
1.8 Risk Assessment and Mitigation Plan	8
1.9 Key Performance Indicators (KPIs)	8
2. Literature Review	**9**
2.1 Introduction	9
2.2 DevOps and Continuous Integration/Continuous Delivery (CI/CD)	9
2.3 Containerization and Orchestration	9
2.4 Infrastructure as Code and Configuration Management	9
2.5 Monitoring and Reliability	10
2.6 Reverse Proxy, Auto-Scaling, and Rollback	10

2.7 Feedback and Evaluation	10
2.8 Suggested Improvements	10
2.9 Final Grading Criteria	11
3. Requirements Gathering	**11**
3.1 Stakeholder Analysis	11
3.2 User Stories	12
3.3 Use Cases	12
3.4 Functional Requirements	13
3.5 Non-Functional Requirements	13

PART 1: PROJECT PLANNING & MANAGEMENT

1.1 Project Proposal

1.1.1 Project Overview

Project Title: Intelligent Traffic Management Dashboard
The *Intelligent Traffic Management Dashboard* is a DevOps-driven project that automates the deployment, scaling, and real-time monitoring of an urban traffic telemetry application. The platform collects simulated traffic data from IoT sensor streams and CSV log feeds, processes congestion metrics, and presents real-time actionable insights on an intuitive web dashboard. By integrating CI/CD, containerization, infrastructure automation, reverse proxying, and comprehensive monitoring, the project eliminates deployment inconsistencies, enhances reliability, and ensures elastic scalability.

1.1.2 Problem Statement

Traditional traffic management and deployment infrastructure face significant operational challenges, including manual deployment overhead, inconsistent development and production environments, difficult scaling under sudden traffic load spikes, and lack of real-time visibility into application health. Municipalities and engineering teams require a modern DevOps workflow to automate deployments, continuously monitor system performance, provision infrastructure declaratively, and support rapid recovery from failed releases.

1.1.3 Proposed Solution

The proposed solution establishes a fully automated DevOps pipeline. **Jenkins** retrieves code from **Git**, executes automated validation, builds **Docker** container images for the traffic data collection and dashboard web services, and deploys them to a **Kubernetes** cluster. **Terraform** provisions necessary AWS cloud infrastructure (EC2, S3, Security Groups), while **Ansible** automates node configuration management. **Nginx** handles traffic routing as a reverse proxy, and **Prometheus** with **Grafana** provides system metrics collection and real-time visual monitoring. Automated rollback strategies ensure high system availability.

1.1.4 Expected Outcome

The expected outcome is a production-grade, maintainable traffic monitoring platform with automated deployment pipelines, horizontal pod autoscaling, high availability, sub-second telemetry ingestion latency, and comprehensive operational dashboards.

1.2 Project Objectives

The primary objective is to implement an automated, scalable, and reliable deployment workflow for a multi-service traffic management dashboard. The specific objectives are to:

1. Automate build, testing, and deployment through Jenkins CI/CD pipelines.
2. Containerize traffic collector and dashboard web services using multi-stage Dockerfiles.
3. Deploy and scale containerized workloads dynamically using Kubernetes (HPA).

4. Provision AWS cloud infrastructure declaratively using Terraform.
5. Automate server setup, package installation, and node configuration using Ansible playbooks.
6. Configure Nginx for reverse-proxy routing and secure ingress.
7. Monitor system and infrastructure metrics using Prometheus and Grafana dashboards.
8. Implement and test an automated rollback procedure for failed updates.
9. Evaluate deployment duration, pipeline success, response time, scaling, and recovery metrics.

1.3 Project Scope

1.3.1 In-Scope Activities

- Developing a traffic data collection microservice (CSV ingestion and simulated IoT sensor HTTP streams).
- Developing a real-time web dashboard displaying traffic congestion, peak hours, and system alerts.
- Creating Dockerfiles and container configurations for application services.
- Implementing Jenkins declarative CI/CD pipelines.
- Deploying services to Kubernetes and configuring health checks (Liveness/Readiness) and Horizontal Pod Autoscaling (HPA).
- Preparing Terraform configurations for selected AWS resources (EC2, S3, Security Groups, VPC).
- Creating Ansible playbooks for automated server provisioning and configuration.
- Configuring Nginx reverse-proxy routing.
- Integrating Prometheus metrics collection and Grafana visualization dashboards.
- Executing deployment, scaling, fault recovery, and rollback tests.
- Maintaining source code and technical documentation.

1.3.2 Out-of-Scope Activities

- Machine Learning / AI predictive traffic forecasting models.
- Physical deployment on physical IoT hardware or hardware traffic cameras.
- Production payment card processing or enterprise commercial billing.
- Multi-region active-active cloud disaster recovery architectures.

1.3.3 Assumptions and Constraints

The project relies on the team's available computing resources, technical skills, and DEPI educational AWS tier limits. Synthetic traffic payloads and CSV logs will simulate real-world IoT sensor streams.

1.3.4 Main Scope Deliverables

- Traffic collection service and real-time web dashboard application code.
- Multi-stage Dockerfiles and container images.

- Jenkins pipeline definitions (`Jenkinsfile`).
- Kubernetes manifests (Deployments, Services, ConfigMaps, HPA).
- Terraform infrastructure scripts and Ansible playbooks.
- Nginx configuration files.
- Prometheus metrics rules and Grafana dashboard JSONs.
- Tested rollback strategy and comprehensive technical documentation.

1.4 Project Plan and Timeline

1.4.1 Project Plan Overview

The project follows a 6-week schedule across 6 phases, covering requirements, microservice development, containerization, CI/CD, IaC, monitoring, and final testing.

1.4.2 Project Timeline

Phase	Main Activities	Duration	Expected Output
1. Planning	Scope, requirements, roles, risk assessment, and KPIs	Week 1	Planning documentation
2. App Development	Develop traffic collector service and web dashboard	Week 2	Functional application microservices
3. Containerization	Create and optimize Dockerfiles and images	Week 3	Tested container images
4. CI/CD & K8s	Build Jenkins pipelines and Kubernetes manifests	Week 4	Automated pipeline & K8s cluster release
5. Infrastructure & Monitoring	Write Terraform, Ansible, Nginx, Prometheus, Grafana	Week 5	AWS IaC & monitoring dashboards
6. Testing & Finalization	Load testing, HPA, rollback validation, final report	Week 6	Verified platform & final report

1.4.3 Gantt Chart

Activity	W1	W2	W3	W4	W5	W6
Planning and Requirements	■					
Application Development		■				
Docker Containerization			■			
Jenkins CI/CD Pipeline				■		
Kubernetes Deployment				■	■	
Terraform and Ansible					■	
Nginx Configuration					■	
Monitoring Setup					■	■
Testing and Rollback						■
Documentation and Presentation	■				■	■

1.5 Milestones and Deliverables

ID	Milestone	Target	Completion Criteria
M1	Planning and Requirements	Week 1	Scope, requirements, roles, risks, and KPIs documented.
M2	Application & Containers	Weeks 2-3	Traffic collector & web dashboard run successfully in Docker.
M3	CI/CD Pipeline	Week 4	Jenkins automates build, test, and container packaging.
M4	Kubernetes Deployment	Weeks 4-5	Workloads deployed on K8s with verified autoscaling (HPA).
M5	Infrastructure & Monitoring	Week 5	Terraform, Ansible, Nginx, Prometheus, & Grafana configured.
M6	Testing & Rollback	Week 6	Automated rollback scenario tested and load benchmarks verified.
M7	Final Submission	Weeks 5-6	Final report, test evidence, and presentation approved.

1.6 Task Assignment and Roles

1.6.1 Team Structure

The project team consists of five engineering members with assigned DevOps lifecycle roles as follows:

Team Member	Proposed Role	Main Responsibilities
Eng. Marwan Elmadawi	Project Coordinator & Infrastructure / IaC Lead	Coordinate sprint tasks, maintain technical docs, write Terraform configurations for AWS EC2/S3, and craft Ansible playbooks for server setup.
Eng. Mohamed Mekhemar	CI/CD & Automation Lead	Construct Jenkins declarative pipelines, manage webhook integrations, enforce automated unit testing, and automate image build/tag workflows.
Eng. Nagat Ali	Kubernetes & Networking Lead	Architect Kubernetes manifests (Deployments, ClusterIP/NodePort Services, ConfigMaps, Secrets, HPA) and configure Nginx reverse proxy routing.
Eng. Asmaa Hany	Application & Containerization Lead	Develop Python traffic collector microservice (CSV/IoT streams) and real-time Web Dashboard; craft optimized multi-stage Dockerfiles.
Eng. Mohamed Abdelhamed	Monitoring, Reliability & SRE Lead	Configure Prometheus metric targets (`/metrics`), design Grafana traffic dashboards, configure Alertmanager, and execute deployment rollback testing.

1.6.2 Task Allocation

Task / Deliverable Domain	Primary Owner	Support Members
Planning and requirements	Eng. Marwan Elmadawi	All team members
Application development (Traffic Collector & Dashboard)	Eng. Asmaa Hany	Eng. Mohamed Mekhemar
Docker and container testing	Eng. Asmaa Hany	Eng. Mohamed Mekhemar
Git repository and Jenkins CI/CD pipeline	Eng. Mohamed Mekhemar	All team members
Kubernetes deployment and HPA scaling	Eng. Nagat Ali	Eng. Mohamed Mekhemar, Eng. Mohamed Abdelhamed
Nginx reverse proxy configuration	Eng. Nagat Ali	Eng. Asmaa Hany
Terraform AWS scripts and Ansible playbooks	Eng. Marwan Elmadawi	Eng. Nagat Ali, Eng. Mohamed Mekhemar
Prometheus metrics & Grafana dashboards	Eng. Mohamed Abdelhamed	Eng. Nagat Ali
Integration, load, and rollback testing	Eng. Mohamed Mekhemar	Eng. Nagat Ali, Eng. Asmaa Hany, Eng. Mohamed Abdelhamed
Documentation, report compiling, & presentation	Eng. Marwan Elmadawi	All team members

1.7 Resource Allocation

Human Resources: 5 DevOps track engineers contributing across development, IaC, CI/CD, orchestration, SRE, and documentation.
Software & Tools: Git, Docker, Jenkins, Kubernetes, Terraform, Ansible, Nginx, Prometheus, Grafana, Python/Flask.
Infrastructure & Cost Management: AWS Cloud EC2 (`t2.micro`), S3 storage, local K3s/Minikube clusters for development testing. AWS budget alerts enforced at $5 to avoid cost overruns.

1.8 Risk Assessment and Mitigation Plan

ID	Risk Event Description	Likelihood	Impact	Mitigation & Contingency Strategy
R1	Integration friction between DevOps tools (Jenkins-K8s API)	Medium	High	Use ServiceAccount tokens with RBAC permissions; test stages incrementally in staging.
R2	AWS Cloud Cost Overrun / Budget Limit	Medium	High	Set up AWS Billing Alerts via Terraform; utilize Free Tier EC2; execute `terraform destroy` when idle.
R3	CI/CD Pipeline Failure or Broken Release	Medium	High	Validate configurations; tag Docker images with Git commit SHA; enable `kubectl rollout undo` on failure.
R4	Simulated IoT Ingestion Spike / Processing Bottleneck	Low	Medium	Implement asynchronous in-memory queueing in Python; enable Kubernetes HPA auto-scaling.
R5	Exposure of Cloud Credentials in Version Control	Medium	High	Enforce `git-secrets` pre-commit hooks; store credentials in Jenkins Credentials Manager & K8s Secrets.

1.9 Key Performance Indicators (KPIs)

KPI Category	Key Performance Indicator	Target Threshold	Measurement Method
Deployment Duration	CI/CD Pipeline Build & Deploy Time	< 5 Minutes	Measured from Git push trigger to successful deployment in Jenkins pipeline logs.
Pipeline Success	CI/CD Pipeline Execution Success Rate	≥ 95%	Successful pipeline runs / total pipeline triggers * 100.
System Latency	Web Dashboard API Response Time (P95)	≤ 300 ms	Measured via Prometheus histogram metric (`http_request_duration_seconds`).
Availability	Application Service Uptime SLA	≥ 99.9%	Monitored via Kubernetes Liveness probes and Prometheus availability metrics.
Fault Recovery	Automated Rollback Duration	< 60 Seconds	Time elapsed between failed release detection and stable version restoration.

PART 2: LITERATURE REVIEW

2.1 Introduction

DevOps is a modern software engineering culture and methodology that unifies software development (Dev) and IT operations (Ops). It emphasizes continuous automation, collaborative workflows, rapid deployment cycles, and continuous monitoring. The *Intelligent Traffic Management Dashboard* applies these principles to deliver a highly reliable, containerized traffic telemetry system.

2.2 DevOps and Continuous Integration/Continuous Delivery (CI/CD)

Continuous Integration (CI) requires developers to commit code frequently into a shared repository, triggering automated compilation, static analysis, and unit testing to detect defects early. Continuous Delivery/Deployment (CD) automates the delivery of validated artifacts to staging and production environments. In this project, **Jenkins** interacts with **Git** repositories to automate container builds, testing, and Kubernetes cluster updates.

2.3 Containerization and Orchestration

Docker Containerization: Encapsulates application code and its dependencies into lightweight, isolated containers, ensuring software runs consistently across development, staging, and AWS production environments. Multi-stage builds are used to minimize image size.
Kubernetes Orchestration: Manages container workloads dynamically. Kubernetes automates container deployment, self-healing pod restarts, internal load balancing, and scaling via the Horizontal Pod Autoscaler (HPA) based on real-time traffic volume.

2.4 Infrastructure as Code and Configuration Management

Terraform (IaC): Enables declarative cloud infrastructure provisioning. Terraform defines AWS EC2 instances, S3 storage buckets, VPC networking, and Security Groups in version-controlled configuration files.
Ansible: Provides agentless configuration management over SSH to automate package installation, Docker runtime setup, and Kubernetes node initialization across provisioned servers.

2.5 Monitoring and Reliability

Observability is critical for maintaining uptime. **Prometheus** collects metrics from application endpoint endpoints (`/metrics`) and Kubernetes nodes at regular scrape intervals. **Grafana** queries Prometheus time-series data to present visual dashboards showing traffic ingestion throughput, system latency, container memory/CPU utilization, and alert statuses.

2.6 Reverse Proxy, Auto-Scaling, and Rollback

Nginx Reverse Proxy: Directs client HTTP traffic to appropriate Kubernetes ingress endpoints, providing SSL termination and request rate limiting.
Auto-Scaling & Rollback: Kubernetes HPA scales container pods up or down depending on CPU/memory load. Automated rollback mechanisms (`kubectl rollout undo`) revert deployments to previous stable releases if health check probes fail.

2.7 Feedback and Evaluation

Feedback from DEPI technical supervisors and automated CI/CD pipeline test reports guide continuous system refinements throughout development.

2.8 Suggested Improvements

Future enhancements include integrating SonarQube static code quality analysis, implementing HashiCorp Vault for secrets management, and adding multi-region disaster recovery.

2.9 Final Grading Criteria

Assessment Category	Evidence & Deliverables Prepared
Documentation (20%)	Thorough Phase 1 report detailing architecture, timeline, risk matrix, user stories, and requirements.
Implementation (20%)	Working Python traffic collector, web dashboard, optimized Dockerfiles, and clean source code.
Automation & Infrastructure (20%)	Validated Jenkins declarative pipelines, Kubernetes manifests, Terraform IaC, and Ansible playbooks.
Monitoring & Reliability (15%)	Prometheus metric scraping, Grafana dashboards, HPA auto-scaling, and verified automated rollback scenario.
Testing & Evaluation (15%)	Recorded test results, pipeline execution logs, latency benchmarks, and disaster recovery evidence.
Presentation & Demonstration (10%)	Clear technical explanation, tool integration demonstration, and live presentation.

PART 3: REQUIREMENTS GATHERING

3.1 Stakeholder Analysis

Stakeholder	Role and Interest	Main Requirements
City Traffic Operators	Monitor city traffic congestion, view bottlenecks, manage alerts.	Real-time visual dashboard, congestion heatmaps, peak time analytics, and clear bottleneck warnings.
Software Developers	Develop application logic, update traffic collector services.	Automated CI testing, containerized local dev environment, and detailed pipeline build logs.
DevOps Engineers	Manage infrastructure, CI/CD pipelines, scaling, monitoring.	Declarative IaC scripts, automated deployments, Prometheus monitoring, and zero-downtime rollback capabilities.
Project Supervisor (Eng. Mena Magdy)	Evaluate technical progress, quality, and DEPI compliance.	Complete technical documentation, robust tool integration, test evidence, and project adherence.

3.2 User Stories

US-01: Browse Real-Time Traffic Congestion (Operator)
As a City Traffic Operator, I want to view a real-time dashboard of traffic congestion so that I can identify road bottlenecks instantly.
Acceptance Criteria: Dashboard updates every 5s; displays color-coded congestion levels (Green/Yellow/Red); shows historical peak times.

US-02: Automated Ingestion of CSV/IoT Data (Data Service)
As a Data Engineer, I want the collection service to ingest CSV logs and IoT streams automatically so that traffic metrics remain accurate.
Acceptance Criteria: Ingests JSON/CSV payloads without corruption; logs invalid records; updates datastore in < 1s.

US-03: Automated Pipeline Deployment (DevOps Engineer)
As a DevOps Engineer, I want code pushes to Git to trigger Jenkins build, test, and Kubernetes deployment automatically.
Acceptance Criteria: Webhook triggers Jenkins pipeline; failed tests halt build; Docker images tagged with SHA and deployed to K8s.

US-04: Auto-Scaling & Health Recovery (SRE)
As an SRE Engineer, I want Kubernetes to autoscale collector pods during traffic load spikes to maintain uptime.
Acceptance Criteria: HPA scales pods when CPU exceeds 70%; Liveness probes restart crashed pods; auto-scales back down when traffic subsides.

3.3 Use Cases

Use Case ID & Title	Primary Actor	Main Flow & Postconditions
UC-01: Ingest Telemetry & Render Dashboard	Traffic Operator / IoT Service	1. IoT service posts JSON traffic payload. 2. Ingestion service parses and updates datastore. 3. Operator views web dashboard showing real-time traffic maps.
UC-02: Automated CI/CD Deployment	Developer / Jenkins	1. Developer pushes code to Git repo. 2. Jenkins pipeline triggers build, unit tests, and Docker image packaging. 3. Jenkins deploys updated manifests to Kubernetes cluster.
UC-03: Automated Rollback on Failure	Jenkins / Kubernetes	1. Updated release deployed to K8s. 2. Readiness probes fail 3 consecutive checks. 3. Pipeline catches failure and runs `kubectl rollout undo` to restore previous build.
UC-04: Monitor Metrics & Alerting	Prometheus / Grafana	1. Prometheus scrapes `/metrics` endpoints. 2. Grafana visualizes latency and memory metrics. 3. Alert triggers if pod memory or HTTP 5xx errors breach thresholds.

3.4 Functional Requirements

- **FR-01:** The system shall ingest simulated traffic data from IoT HTTP streams and CSV file logs.
- **FR-02:** The system shall calculate traffic congestion density and categorize status into Green, Yellow, and Red thresholds.
- **FR-03:** The web dashboard shall render real-time visual graphs of traffic density, peak hours, and sensor alerts.
- **FR-04:** Application microservices shall be containerized using isolated multi-stage Dockerfiles.
- **FR-05:** Jenkins shall automate code checkout, unit testing, Docker image creation, and Kubernetes deployment execution.
- **FR-06:** Nginx shall route client HTTP traffic to backend application services as a reverse proxy.
- **FR-07:** Prometheus shall scrape operational metrics, and Grafana shall render interactive system monitoring dashboards.

3.5 Non-Functional Requirements

- **NFR-01 (Performance):** Dashboard rendering time shall be under 500ms; API latency P95 shall remain below 300ms.
- **NFR-02 (Scalability):** Kubernetes HPA shall autoscale ingestion pods from 2 to 10 replicas based on CPU load.
- **NFR-03 (Reliability):** Service availability SLA shall meet or exceed 99.9% uptime backed by Kubernetes self-healing probes.

- **NFR-04 (Security):** Sensitive cloud credentials shall not be committed to version control; Nginx shall enforce HTTPS/TLS.
- **NFR-05 (Maintainability):** Cloud infrastructure shall be 100% automated using Terraform scripts and Ansible playbooks.